



The syntax of the aggregate functions

- **AVG (expression)**
- **SUM (expression)**
- **MIN (expression)**
- **MAX (expression)**
- **COUNT (expression)**
- **COUNT (*)**

AVG (expression)

Computes the average (arithmetic mean) of all the non-null input values.

SUM (expression)

Computes the sum of the non-null input values.

MIN (expression)

Computes the minimum of the non-null input values. Available for any numeric, string, date/time, or enum type, as well as inet, interval, money, oid, pg_lsn, tid, xid8, and arrays of any of these types.

MAX (expression)

Computes the maximum of the non-null input values.

COUNT (expression)

Computes the number of input rows in which the input value **is not null**.

COUNT (*)

Computes the number of input rows. **This includes NULL values.**

A Simple Use of count (*)

```
SELECT count(*)  
FROM chap_09_examples.pls_fy2018_libraries;  
9261
```

```
SELECT count(*)  
FROM chap_09_examples.pls_fy2017_libraries;  
9245
```

```
SELECT count(*)  
FROM chap_09_examples.pls_fy2016_libraries;  
9252
```

This includes NULL values.

A Simple Use of count (phone)

```
SELECT count(phone)
FROM chap_09_examples.pls_fy2018_libraries;
```

9261

Returns the number of rows that are **not NULL**.

This means every row in the **phone** column has a value.

Use of count (DISTINCT phone)

```
SELECT count(libname)
FROM chap_09_examples.pls_fy2018_libraries;
9261
```

```
SELECT count(DISTINCT libname)
FROM chap_09_examples.pls_fy2018_libraries;
8478
```

There are quite a few libraries with duplicate names.
Ten libraries are named OXFORD PUBLIC LIBRARY.

All those Oxford Libraries

```
-- see location of every Oxford Public Library
SELECT libname, city, stabr
FROM chap_09_examples.pls_fy2018_libraries
WHERE libname = 'OXFORD PUBLIC LIBRARY'
order by stabr;
```

libname	city	stabr
OXFORD PUBLIC LIBRARY	OXFORD	AL
OXFORD PUBLIC LIBRARY	OXFORD	CT
OXFORD PUBLIC LIBRARY	OXFORD	IA
OXFORD PUBLIC LIBRARY	OXFORD	IN
OXFORD PUBLIC LIBRARY	OXFORD	KS
OXFORD PUBLIC LIBRARY	OXFORD	MI
OXFORD PUBLIC LIBRARY	OXFORD	NE
OXFORD PUBLIC LIBRARY	OXFORD	NJ
OXFORD PUBLIC LIBRARY	OXFORD	PA
OXFORD PUBLIC LIBRARY	OXFORD	WI

Maximum and Minimum Values Using `max()` and `min()`

```
SELECT max(visits), min(visits)
FROM chap_09_examples.pls_fy2018_libraries;

max      min
16686945  -3
```

The creators of the library survey are employing a common but potentially problematic convention in data collection by placing a negative number or some artificially high value in a column to indicate some condition.

In this case, negative values in number columns indicate the following:

- A value of -1 indicates a “nonresponse” to that question.
- A value of -3 indicates “not applicable” and is used when a library agency has closed either temporarily or permanently.

Looking at all Three Years

```
SELECT 2018 as year, max(visits), min(visits)
FROM chap_09_examples.pls_fy2018_libraries
union
SELECT 2017, max(visits), min(visits)
FROM chap_09_examples.pls_fy2017_libraries
union
SELECT 2016, max(visits), min(visits)
FROM chap_09_examples.pls_fy2017_libraries;
```

year	max	min
2,018	16,686,945	-3
2,016	17,179,330	-3
2,017	17,179,330	-3

Aggregating Data Using GROUP BY

```
SELECT stabr  
FROM chap_09_examples.pls_fy2018_libraries  
GROUP BY stabr  
ORDER BY stabr;
```

When you use the **GROUP BY** clause with aggregate functions, you can group results according to the values in one or more columns. This allows us to perform operations such as **sum()** or **count()** for each state in the table or for each type of library agency.

```
-- There are 55 in 2018.  
SELECT stabr  
FROM chap_09_examples.pls_fy2018_libraries  
GROUP BY stabr  
ORDER BY stabr;
```

stabr
AK
AL
AR
AS
AZ
CA
...

Grouping by more than one column

```
SELECT city, stabr  
FROM chap_09_examples.pls_fy2018_libraries  
GROUP BY city, stabr  
ORDER BY city, stabr;
```

You're not limited to grouping just one column. In Listing 9-8, we use the **GROUP BY** clause on the 2018 data to specify the `city` and `stabr` columns for grouping.

This says to first group by the `city` name and within each city grouping, group by `stabr`.

```
SELECT city, stabr
FROM chap_09_examples.pls_fy2018_libraries
GROUP BY city, stabr
ORDER BY city, stabr;
```

city	stabr
ABBEVILLE	AL
ABBEVILLE	LA
ABBEVILLE	SC
ABBOTSFORD	WI
ABERDEEN	ID
ABERDEEN	SD
ABERNATHY	TX
ABILENE	KS
ABILENE	TX
... ..	

```
SELECT city, stabr
FROM chap_09_examples.pls_fy2018_libraries
GROUP BY stabr, city
ORDER BY stabr, city;
```

city	stabr
AKIACHAK	AK
AKIAK	AK
ANCHORAGE	AK
ANCHOR POINT	AK
ANDERSON	AK
ANIAK	AK
ANVIK	AK
BARROW	AK
BETHEL	AK

Combining GROUP BY with count ()

```
SELECT stabr, count(*)  
FROM chap_09_examples.pls_fy2018_libraries  
GROUP BY stabr  
ORDER BY count(*) DESC;
```

stabr	count
NY	756
IL	623
TX	560
IA	544
PA	451
MI	398
WI	381
... ..	

The tables contains a code indicating whether the agency's address changed in the last year. The values in **stataddr** are as follows:

- 00 No change from last year
- 07 Moved to a new location
- 15 Minor address change

```
SELECT stabr, stataddr, count(*)  
FROM chap_09_examples.pls_fy2018_libraries  
GROUP BY stabr, stataddr  
ORDER BY stabr, stataddr;
```

00 No change from last year
07 Moved to a new location
15 Minor address change

stabr	stataddr	count
AK	0	82
AL	0	220
AL	15	1
AL	7	3
AR	0	58
AR	15	1
AR	7	1
AS	0	1
AZ	0	88


```
SELECT city, stabr, count(*)  
FROM chap_09_examples.pls_fy2018_libraries  
GROUP BY city, stabr  
ORDER BY count(*) DESC;
```

city	stabr	count
PITTSBURGH	PA	16
ROCHESTER	NY	6
AUSTIN	TX	4
BICKNELL	UT	4
TUCSON	AZ	3
ROANOKE	VA	3
PORTLAND	OR	3

Revisiting sum () to Examine Library Activity

```
SELECT 2018 as year, sum(visits) AS visits_2018
FROM chap_09_examples.pls_fy2018_libraries
WHERE visits >= 0
union
SELECT 2017, sum(visits) AS visits_2017
FROM chap_09_examples.pls_fy2017_libraries
WHERE visits >= 0
union
SELECT 2016, sum(visits) AS visits_2016
FROM chap_09_examples.pls_fy2016_libraries
WHERE visits >= 0;
```

```
SELECT 2018 as year, sum(visits) AS visits_2018
FROM chap_09_examples.pls_fy2018_libraries
WHERE visits >= 0
```

union

```
SELECT 2017, sum(visits) AS visits_2017
FROM chap_09_examples.pls_fy2017_libraries
WHERE visits >= 0
```

union

```
SELECT 2016, sum(visits) AS visits_2016
FROM chap_09_examples.pls_fy2016_libraries
WHERE visits >= 0;
```

year	visits_2018
2,016	1,355,648,985
2,017	1,319,803,987
2,018	1,292,348,697

```
SELECT sum(pls18.visits) AS visits_2018,  
       sum(pls17.visits) AS visits_2017,  
       sum(pls16.visits) AS visits_2016  
FROM chap_09_examples.pls_fy2018_libraries pls18  
  JOIN chap_09_examples.pls_fy2017_libraries pls17  
    ON pls18.fscskey = pls17.fscskey  
  JOIN chap_09_examples.pls_fy2016_libraries pls16  
    ON pls18.fscskey = pls16.fscskey  
WHERE pls18.visits >= 0  
      AND pls17.visits >= 0  
      AND pls16.visits >= 0;
```

visits_2018	visits_2017	visits_2016
1,278,148,838	1,319,325,375	1,355,078,382

```
SELECT sum(pls18.wifisess) AS wifi_2018,  
       sum(pls17.wifisess) AS wifi_2017,  
       sum(pls16.wifisess) AS wifi_2016  
FROM chap_09_examples.pls_fy2018_libraries pls18  
JOIN chap_09_examples.pls_fy2017_libraries pls17  
  ON pls18.fscskey = pls17.fscskey  
JOIN chap_09_examples.pls_fy2016_libraries pls16  
  ON pls18.fscskey = pls16.fscskey  
WHERE pls18.wifisess >= 0  
      AND pls17.wifisess >= 0  
      AND pls16.wifisess >= 0;
```

wifi_2018	wifi_2017	wifi_2016
349,767,271	311,336,210	234,926,102

Grouping Visit Sums by State

```
SELECT pls18.stabr,  
       sum(pls18.visits) AS visits_2018,  
       sum(pls17.visits) AS visits_2017,  
       sum(pls16.visits) AS visits_2016,  
       round( (sum(pls18.visits::numeric) - sum(pls17.visits)) /  
              sum(pls17.visits) * 100, 1 ) AS chg_2018_17,  
       round( (sum(pls17.visits::numeric) - sum(pls16.visits)) /  
              sum(pls16.visits) * 100, 1 ) AS chg_2017_16  
FROM chap_09_examples.pls_fy2018_libraries pls18  
  JOIN chap_09_examples.pls_fy2017_libraries pls17  
    ON pls18.fscskey = pls17.fscskey  
  JOIN chap_09_examples.pls_fy2016_libraries pls16  
    ON pls18.fscskey = pls16.fscskey  
WHERE pls18.visits >= 0  
      AND pls17.visits >= 0  
      AND pls16.visits >= 0  
GROUP BY pls18.stabr  
ORDER BY chg_2018_17 DESC;
```

stabr	visits_2018	visits_2017	visits_2016	chg_2018_17	chg_2017_16
SD	3,824,804	3,699,212	3,722,376	3.4	-0.6
MT	4,332,900	4,215,484	4,298,268	2.8	-1.9
FL	68,423,689	66,697,122	70,991,029	2.6	-6
ND	2,216,377	2,162,189	2,201,730	2.5	-1.8
ID	8,179,077	8,029,503	8,597,955	1.9	-6.6
DC	3,632,539	3,593,201	3,930,763	1.1	-8.6
ME	6,746,380	6,731,768	6,811,441	0.2	-1.2
NH	7,045,010	7,028,799	7,236,565	0.2	-2.9

Filtering an Aggregate Query Using **HAVING**

To filter the results of aggregate functions, we need to use the **HAVING** clause.

You've used **WHERE** clause for filtering, but aggregate functions, such as **sum ()**, can't be used within a **WHERE** clause because they operate at the row level, and aggregate functions work across rows.

The **HAVING** clause places conditions on groups created by aggregating.


```
SELECT pls18.stabr,  
       sum(pls18.visits) AS visits_2018,  
       sum(pls17.visits) AS visits_2017,  
       sum(pls16.visits) AS visits_2016,  
       round( (sum(pls18.visits::numeric) - sum(pls17.visits)) /  
              sum(pls17.visits) * 100, 1 ) AS chg_2018_17,  
       round( (sum(pls17.visits::numeric) - sum(pls16.visits)) /  
              sum(pls16.visits) * 100, 1 ) AS chg_2017_16  
FROM chap_09_examples.pls_fy2018_libraries pls18  
   JOIN chap_09_examples.pls_fy2017_libraries pls17  
       ON pls18.fscskey = pls17.fscskey  
   JOIN chap_09_examples.pls_fy2016_libraries pls16  
       ON pls18.fscskey = pls16.fscskey  
WHERE pls18.visits >= 0  
      AND pls17.visits >= 0  
      AND pls16.visits >= 0  
GROUP BY pls18.stabr  
HAVING sum(pls18.visits) > 50000000  
ORDER BY chg_2018_17 DESC;
```

stabr	visits_2018	visits_2017	visits_2016	chg_2018_17	chg_2017_16
FL	68,423,689	66,697,122	70,991,029	2.6	-6
NY	97,921,323	100,012,193	103,081,304	-2.1	-3
CA	146,656,984	151,056,672	155,613,529	-2.9	-2.9
IL	63,466,887	66,166,082	67,336,230	-4.1	-1.7
OH	68,176,967	71,895,854	74,119,719	-5.2	-3
TX	66,168,387	70,514,128	70,975,901	-6.2	-0.7